Member 1

1. Read the proposal and blueprint and list all backend, security, administration and deployment responsibilities.
2. Inspect the repository structure and confirm where configuration, routes, controllers, services, jobs and SQL belong.
3. Install Git, Node.js, npm, PostgreSQL and a database administration tool on the development computer.
4. Clone the repository, check out the shared development branch and use a feature branch for each change.
5. Agree with all members which features belong in the initial release and record unresolved questions with Member 6.
6. Confirm the pharmacy-computer server and local-network deployment model with the pharmacy owner.
7. Agree Admin and Pharmacist permissions for every protected operation and share the permission matrix.
8. Agree the pharmacy timezone, date-only expiry interpretation and treatment of medicine expiring on the current date.
9. Agree available-stock rules for expired, inactive, empty and returned batches with Members 3 and 4.
10. Agree currency precision, rounding, discount scope and authoritative server pricing with Members 4 and 6.
11. Agree API resource names, HTTP methods, status codes, pagination and search conventions with all API owners.
12. Agree JSON field names, error shape and authentication headers before members connect their screens.
13. Assign ownership for every endpoint, including dashboard reports, administration and password changes.
14. Initialize the Node.js package inside medixmart-backend and configure development and start commands.
15. Install the selected Express, PostgreSQL, environment, CORS, password-hashing, JWT and scheduling dependencies.
16. Create a tracked environment example containing placeholders for database, server, token and messaging settings.
17. Create the private local environment file and verify that Git ignores it and credential files.
18. Validate required environment variables at startup and fail clearly when configuration is incomplete.
19. Create the PostgreSQL connection pool in src/config and verify database connectivity.
20. Use a restricted application database account and keep administrative credentials separate.

Member 1

21. Create the Express entry point and register JSON request parsing with a reasonable payload limit.
22. Configure CORS for the agreed clients without unnecessarily exposing authenticated browser requests.
23. Implement the health endpoint and distinguish server availability from database readiness as agreed.
24. Connect the server to Member 3's schema and verify a simple database query succeeds.
25. Create an async handler helper so rejected controller promises reach the shared error handler.
26. Create centralized errors with consistent status codes and safe response messages.
27. Add a not-found handler for unknown API routes and a final error-handling middleware.
28. Prevent stack traces, database passwords and internal SQL details from reaching production clients.
29. Add useful startup, request-failure and job logs without recording passwords, tokens or private customer details.
30. Document how developers configure, start and stop the backend locally.
31. Agree the users schema and role constraints with Member 3.
32. Create a safe admin-seeding script that hashes the supplied password and avoids duplicate accounts.
33. Implement login input validation and use parameterized SQL to look up the user.
34. Compare login passwords with bcrypt and reject incorrect credentials with a consistent authentication response.
35. Reject disabled users during login and protected requests.
36. Issue signed tokens containing the user identity and role with the agreed expiration policy.
37. Create Bearer-token middleware that rejects missing, malformed, expired and invalid tokens.
38. Create role-checking middleware and apply it to every restricted backend action.
39. Define logout and session invalidation behavior and explain it to Member 2.
40. Implement the password-change endpoint, checking the current password and hashing the replacement.
41. Define and enforce how password changes affect existing sessions.
42. Implement protected user-list, user-create, user-update and user-disable endpoints for administrators.
43. Validate username uniqueness, required fields and allowed role values.
44. Prevent administrators from accidentally removing the final usable administrator account.
45. Implement the agreed settings read/update endpoints and validate configurable values.
46. Coordinate settings and audit-table changes with Member 3 before using them.
47. Record agreed administrative actions in the audit trail without logging secrets.

Member 1

48. Keep production secrets outside settings responses and restrict audit access to authorized users.
49. Review all new routes to ensure authentication and role checks are actually applied.
50. Write backend tests for login, tokens, roles, disabled users, password changes and administration.
51. Share working login, protected-request and error examples with Member 2.
52. Help Member 2 integrate secure token storage, logout and expired-session handling.
53. Review Members 3, 4, 5 and 6's backend changes for validation, SQL safety and contract consistency.
54. Check that database clients are released after both successful and failed requests.
55. Check that multi-step sale and repayment operations use one connection and one transaction.
56. Review sale batch locks, lock ordering and rollback behavior with Member 4.
57. Coordinate fixes for simultaneous sales and conflicting repayments with the relevant owners.
58. Help members diagnose incorrect API URLs, network access, request JSON and missing authorization headers.
59. Publish a short Flutter-to-API guide containing a successful request and an error-handling example.
60. Implement or review the SQL function that finds stocked batches within the seven-month expiry window.
61. Distinguish return-soon stock from stock that has already expired using the agreed boundary rules.
62. Include enough batch identity in alerts to trace the affected stock correctly.
63. Prevent repeated expiry checks from creating duplicate active alerts for the same condition.
64. Create the expiry job under src/jobs and make its timezone and schedule configurable.
65. Run an expiry check during startup so a missed scheduled run is recovered.
66. Prevent overlapping executions of the same job and record successful and failed runs.
67. Define how resolved expiry alerts are treated when stock is returned, depleted or corrected.
68. Implement the low-stock trigger or service call inside the successful stock-changing transaction.
69. Calculate available medicine stock consistently with the inventory and sales APIs.
70. Create one active low-stock alert when stock reaches or falls below its reorder level.
71. Define how a low-stock alert is resolved after replenishment and recreated after a later shortage.
72. Recheck low-stock conditions after relevant stock receipts, returns or agreed stock adjustments.
73. Test threshold equality, repeated sales, empty stock and replenishment with Members 3 and 6.
74. Review Member 5's birthday job configuration, delivery logging and duplicate prevention.
75. Ensure development-only messaging test routes are unavailable in production.

Member 1

76. Add graceful shutdown that stops scheduled work, closes the HTTP server and releases database connections.
77. Add security headers and rate limits for login and other agreed sensitive endpoints.
78. Review input size limits, error leakage, secret handling and dependency issues before release.
79. Review query performance with Member 3 and add justified indexes based on measured queries.
80. Re-run security and integration checks after changing middleware or database functions.
81. Review open pull requests, request corrections and merge approved backend changes through the agreed workflow.
82. Create a database-backup script using the restricted operational setup agreed with the pharmacy.
83. Store backup output outside the repository and restrict access to the backup location.
84. Define backup frequency, retention and failure reporting with the pharmacy owner.
85. Restore a backup into a separate test database and verify records, balances and relationships.
86. Write installation, configuration, startup, backup, restore and troubleshooting instructions for Member 6.
87. Check the pharmacy computer's storage, operating system, network and required runtime versions.
88. Install Node.js and PostgreSQL on the pharmacy computer and create the production database and account.
89. Apply the reviewed schema and production configuration without using development passwords or sample accounts.
90. Configure the backend as an automatically starting Windows service or the agreed startup mechanism.
91. Set the stable local server address and allow only the required network/firewall access.
92. Verify the Windows client and an Android device can reach the server on pharmacy Wi-Fi.
93. Coordinate with Members 3 and 4 to import legacy data after the production database is ready.
94. Check the service restarts correctly after reboot and recovers database connectivity as designed.
95. Configure automatic backups on the pharmacy computer and demonstrate restoration from a real backup.
96. Disable development endpoints, remove unnecessary debug output and rotate temporary credentials.
97. Run final access, sale-rollback, concurrency, job and administration checks with Member 6.
98. Fix backend defects identified during pharmacy acceptance testing and provide evidence for retesting.
99. Transfer credentials and operational instructions securely to the authorized pharmacy representative.
100. Hand over configuration notes, service controls, recovery steps and the agreed support contact process.
101. Confirm backend changes are reviewed, tested, merged and documented before project closure.

Member 2

1. Read the proposal and blueprint and identify every client screen and responsive workflow.
2. Inspect the Flutter folder structure and confirm shared screen, model, service, provider and widget locations.
3. Install Flutter, Dart, Git and the tools needed to build Android and Windows applications.
4. Verify Flutter tooling and resolve missing Windows compiler or Android SDK requirements.
5. Clone the repository and use feature branches and pull requests for client changes.
6. Review actual pharmacy workflows, staff terminology, computer resolution and Android device sizes.
7. Agree the navigation map and screen ownership with Members 3, 4, 5 and 6.
8. Create the shared Figma workspace and design reusable colors, typography, spacing, buttons and form controls.
9. Design desktop and mobile login screens with validation, loading and authentication-error states.
10. Design desktop and mobile dashboards, including sales, stock and expiry cards.
11. Design desktop and mobile POS layouts with medicine search, cart, payment and receipt states.
12. Share the designs with the pharmacy and team, collect feedback and revise the agreed flows.
13. Define reusable patterns for tables, mobile lists, search, filters, dialogs and empty states.
14. Define keyboard navigation, visible focus, readable contrast and touch-target requirements.
15. Initialize the Flutter project in medixmart_app while preserving the existing folder structure.
16. Enable Windows and Android targets and verify the unmodified project launches on both.
17. Add the selected state-management, networking, routing and secure-storage packages.
18. Set the application name, theme and shared visual constants.
19. Create the main application entry point and register shared providers.
20. Create configurable API base-address handling for local development and installed deployments.
21. Implement the central API service with JSON encoding, decoding and authorization headers.
22. Handle network failures, invalid responses, timeouts and server validation messages consistently.
23. Create shared typed models for user/authentication responses and coordinate other model fields with feature owners.
24. Implement secure token storage and load a saved session when the app starts.

Member 2

25. Create AuthProvider for login state, current user, role and logout actions.
26. Implement protected routes and redirect unauthenticated users to login.
27. Handle expired or rejected tokens by clearing invalid session state and presenting the login flow.
28. Build the login form with required-field validation and keyboard submit behavior.
29. Disable duplicate login submission while a request is pending.
30. Connect login to Member 1's endpoint and display clear success or failure feedback.
31. Implement logout and remove stored credentials/session tokens according to the agreed policy.
32. Build the password-change screen, including current, new and confirmation fields.
33. Validate password-change input, connect the endpoint and apply the agreed session-reset behavior.
34. Create desktop navigation and mobile navigation for all authorized feature screens.
35. Hide unauthorized actions in the interface while preserving server-enforced authorization.
36. Create reusable loading, retry, error, empty-list and confirmation widgets.
37. Create common text inputs, date pickers, dropdowns, search fields and validation messages.
38. Coordinate shared data models and services so feature owners do not duplicate incompatible implementations.
39. Create MedicineProvider and connect medicine loading, refresh and search behavior.
40. Build a searchable medicine table for desktop and list for mobile.
41. Display medicine name, category, selling price, reorder level and available stock as agreed.
42. Handle empty search results, service failures and large result sets using the agreed pagination behavior.
43. Build the add-medicine form with required name, category, price and reorder-level fields.
44. Validate numeric prices and reorder levels before submitting data.
45. Connect medicine creation and refresh the displayed list after success.
46. Build the edit-medicine form and show server-side duplicate or validation errors.
47. Add safe medicine-deactivation controls for authorized staff and confirm the action before submission.
48. Show inactive medicines appropriately without breaking historical views.
49. Build category list and add-category controls using the agreed category endpoints.

Member 2

50. Refresh category choices after a category is added successfully.
51. Build supplier list and search controls with clear supplier contact details.
52. Build supplier add/edit forms and validate the agreed required fields.
53. Connect supplier endpoints and refresh supplier data after successful changes.
54. Ensure a newly added supplier becomes selectable in Member 4's Receive Stock form.
55. Help Member 4 reuse medicine/supplier selectors and shared validation widgets.
56. Help Member 5 reuse customer forms, search controls and repayment dialogs.
57. Help Member 3 reuse alert filters and supplier-return list layouts.
58. Build dashboard cards for the agreed sales, low-stock, return-soon, expired-stock and credit indicators.
59. Create a seven-day sales chart with meaningful axis labels and currency formatting.
60. Provide dashboard loading, no-data and API-error states.
61. Agree dashboard response fields and reporting boundaries with Members 3 and 6.
62. Replace temporary dashboard values with real alert-summary and report API responses.
63. Reconcile displayed dashboard totals with known test sales and database results.
64. Add dashboard manual refresh and the agreed automatic refresh behavior.
65. Stop refresh timers when the dashboard is disposed and avoid overlapping requests.
66. Make dashboard cards open their corresponding alert, stock, sales or credit views.
67. Refresh affected dashboard state after successful sales, stock receipts and repayments.
68. Build the administrator user-list screen with account status and role visibility.
69. Build authorized user-create, edit, role-change and disable controls.
70. Connect administration actions to Member 1's endpoints and show protected-action errors clearly.
71. Build agreed settings controls, including the server address needed for installed clients.
72. Validate setting values and provide a clear recovery path for an incorrect server address.
73. Add the agreed read-only audit view for administrators if included in the confirmed administration scope.
74. Review every client screen for consistent labels, dates, money formatting and feedback.
75. Check keyboard-only navigation for desktop login, forms, searches and checkout.

Member 2

76. Check touch interaction, scrolling and navigation on a real Android device.
77. Check larger text sizes, form focus, readable errors and safe layout resizing.
78. Fix overflow, clipped text, hidden controls and unusable table layouts at pharmacy screen sizes.
79. Add meaningful widget/unit tests for authentication, route guards, critical forms and shared API error handling.
80. Review other members' Flutter pull requests for consistency, usability and integration defects.
81. Run static analysis and client tests and resolve relevant issues before merging changes.
82. Test reconnecting after the server is unavailable without losing or falsely confirming user work.
83. Test repeated button presses and back navigation during pending requests.
84. Verify printing, sharing, exports and messaging-related packages work on the selected target platforms.
85. Prepare the production application icon and final display name.
86. Set final API configuration and coordinate the pharmacy server address with Member 1.
87. Configure Android permissions and release networking behavior for the approved deployment topology.
88. Store signing material privately and document how authorized maintainers produce future releases.
89. Build the release Windows application and package the complete runtime into an installer.
90. Build the signed release Android APK and identify the release version clearly.
91. Install the Windows release on a clean target computer and verify startup and API access.
92. Install the Android release on a clean device and verify login and main pharmacy workflows.
93. Test upgrade/reinstall behavior and confirm application settings survive or reset as documented.
94. Check the Windows installation does not require development tools on the pharmacy computer.
95. Supply versioned release builds, packaging instructions and final interface screenshots to Member 6.
96. Support pharmacy acceptance testing and fix client defects found by staff.
97. Rebuild and retest affected platform releases after release fixes.
98. Help Member 5 prepare screenshots and demonstrations for staff training.
99. Explain client configuration, login recovery and normal update steps during handover.
100. Confirm all client work is reviewed, tested, merged and represented in the final release.

Member 3

1. Read the proposal and blueprint and identify inventory, supplier, batch, expiry and return requirements.
2. Inspect backend and Flutter folders assigned to inventory and agree boundaries with other members.
3. Install database, API testing and client tools needed for the assigned implementation work.
4. Clone the repository and use feature branches for schema, API and screen changes.
5. Review the pharmacy's medicine names, categories, supplier details, packaging and stock-recording practices.
6. Agree medicine identifiers, name normalization, duplicate rules and deactivation behavior.
7. Agree stock units, received-date rules, expiry boundaries and the available-stock definition with Members 1 and 4.
8. Agree whether received-stock records alone cover purchases or separate order tracking is required by the pharmacy.
9. Confirm the seven-month supplier-return rule and distinguish return candidates from expired stock.
10. Agree whether actual return recording is in scope and how quantities, history and authorization should behave.
11. Draw the initial data relationships with Member 6 before writing the schema.
12. Create the categories table with required name and appropriate uniqueness rules.
13. Create the medicines table with category link, unit price, reorder level and active status.
14. Create the suppliers table with the agreed contact fields.
15. Create the batches table with medicine, supplier, batch number, quantity, cost, received date and expiry date.
16. Create the users table and agree identity, password-hash, role and active-status fields with Member 1.
17. Create the customers table and agree birthday and credit-balance fields with Member 5.
18. Create sales and sale_items tables with precise monetary fields and exact batch links.
19. Create credit_payments and alerts tables with the fields agreed with their feature owners.
20. Add settings, audit and notification-log schema required by the confirmed administration and messaging features.
21. Add supplier-return history schema if actual return recording is included in the confirmed scope.
22. Define primary keys, foreign keys, required fields and safe deletion behavior.
23. Add database checks for valid quantities, prices, reorder levels, payment amounts and allowed values.

Member 3

24. Use appropriate decimal types for money and consistent date/time types.
25. Index batch medicine/expiry lookups and other justified foreign-key or search paths.
26. Write schema scripts in an explicit execution order under medixmart-backend/src/sql.
27. Run the schema scripts on a clean database and fix dependency or creation-order problems.
28. Document schema creation and changes so every developer can recreate the same database.
29. Review schema changes with Members 1, 4, 5 and 6 before dependent code is merged.
30. Create synthetic seed categories, suppliers and medicine records using realistic but nonprivate data.
31. Create multiple batches per medicine with different quantities, received dates and expiry dates.
32. Include stock within and outside the return window, empty batches and already expired batches for testing.
33. Include medicines above, equal to and below their reorder levels.
34. Coordinate seeded users, birthday customers, credit balances and linked sale/payment history with the owners.
35. Check seed totals, foreign keys and financial balances and document how to load or reset test data.
36. Implement protected category-list and category-create endpoints.
37. Validate category names and return consistent duplicate and missing-field errors.
38. Implement medicine list, detail, create and update endpoints.
39. Add medicine search using the agreed query parameter and safe parameterized SQL.
40. Include category, selling price, reorder level and calculated available stock in medicine responses.
41. Validate medicine input, category existence, numeric values and duplicate-name rules.
42. Implement authorized medicine deactivation without deleting historical sale references.
43. Test that inactive medicines cannot accidentally be sold through normal active-item search.
44. Implement supplier list, detail, create, update and agreed search behavior.
45. Validate supplier details and return clear errors for unknown supplier IDs.
46. Share category, medicine and supplier payload examples with Member 2.
47. Implement batch-list filtering by medicine and any agreed supplier or status filters.
48. Sort eligible batch results by expiry with deterministic ordering for equal expiry values.
49. Implement batch creation and verify the referenced medicine and supplier exist.

Member 3

50. Reject invalid quantity, cost, batch details and nonfuture expiry according to the agreed rules.
51. Record received date and preserve separate batch records where expiry differs.
52. Return saved batch details and updated stock information needed by Member 4's form.
53. Define and implement any agreed stock-correction flow with audit history instead of silent quantity edits.
54. Keep stock changes transactional and prevent invalid or negative inventory values.
55. Test category, medicine, supplier and batch validation and authorization with Member 6.
56. Verify medicine stock totals against SQL after receipt, sale, adjustment and return operations.
57. Help Member 4 integrate the Receive Stock form and resolve payload or refresh issues.
58. Implement alerts-list endpoint with newest-first ordering and agreed pagination.
59. Add alert-type and unread filters and include medicine/batch context in responses.
60. Implement the protected mark-alert-read endpoint and validate the alert exists.
61. Implement alert-summary counts required by the dashboard.
62. Provide clear expired-stock and return-soon query behavior using the agreed boundary definitions.
63. Check summary counts against stored alerts and actual eligible batch records.
64. Coordinate alert uniqueness, resolution and reactivation rules with Member 1.
65. Build the Flutter alerts screen with loading, empty, retry and failure states.
66. Show meaningful icons, medicine information, expiry details and unread emphasis.
67. Add All, Expiry and Low Stock filter controls using the API filters.
68. Connect mark-read actions and refresh affected list and summary state.
69. Check alert behavior after sales, replenishment and return operations with Members 1 and 4.
70. Implement the expiring-batches endpoint for the agreed seven-month return-candidate window.
71. Include medicine, batch, supplier, expiry and remaining quantity in return-candidate responses.
72. Add supplier filtering/grouping and consistently exclude empty or already returned stock.
73. Build the supplier-return screen with desktop table and mobile-friendly layout.
74. Add supplier selection, clear expiry/expired indicators and empty/error states.
75. Reconcile the displayed return list against database records and the pharmacy rule.
76. If actual return recording is agreed, implement authorized return submission with positive validated quantities.

Member 3

77. For actual returns, write return history and deduct stock in one transaction with appropriate row locks.
78. For actual returns, prevent repeated submission from deducting the same stock twice.
79. For actual returns, refresh inventory and alerts and preserve sale/receipt history.
80. If actual return recording is not agreed, deliver the candidate list without a misleading stock-changing button.
81. Create meaningful API and inventory tests for duplicates, invalid references, inactive medicines and expiry boundaries.
82. Check transaction rollback and stock integrity for failed receipt, adjustment or return requests.
83. Review measured slow inventory queries with Member 1 and add justified indexes.
84. Update API examples, schema documentation and inventory diagrams with Member 6.
85. Review and merge tested inventory changes through the agreed pull-request workflow.
86. Collect the pharmacy's legacy inventory records with permission and preserve an original copy privately.
87. Define import columns for categories, medicines, suppliers, batches, quantities, costs, prices and expiry dates.
88. Agree duplicate matching, stock units and incomplete-record handling with Member 4.
89. Clean duplicate categories, names and suppliers before importing.
90. Identify missing or ambiguous expiry dates and ask authorized pharmacy staff to resolve them.
91. Dry-run the import tool with Member 4 on a test database and inspect rejected-row reports.
92. Compare imported medicine, supplier and batch counts with source records.
93. Reconcile stock totals and sample batches with the pharmacy's physical or trusted records.
94. Apply approved import corrections and keep a private reconciliation record.
95. Provide reviewed schema scripts and inventory setup notes for Member 1's production installation.
96. Run the approved inventory import after the production database is ready and backed up.
97. Verify production stock, expiry alerts and return candidates on Windows and Android.
98. Demonstrate medicine maintenance, stock receipt, alerts and supplier-return views during staff training.
99. Fix inventory defects found during acceptance testing and support Member 6's retesting.
100. Hand over schema scripts, import mappings, inventory instructions and known operational limitations.
101. Confirm all inventory code, tests and documentation are reviewed and merged before closure.

Member 4

1. Read the proposal and blueprint and identify checkout, stock-receipt, receipt and migration responsibilities.
2. Inspect the backend and Flutter folders used by sales, services, providers, stock forms and receipt generation.
3. Install development and API testing tools and verify the backend and client can run locally.
4. Clone the repository and use a feature branch for each sales or billing change.
5. Observe how pharmacy staff search medicines, determine quantities, collect payment and issue receipts.
6. Confirm stock units, selling prices, rounding and whether discounts belong in the confirmed release.
7. Agree that the server validates prices and calculates the final bill rather than trusting client totals.
8. Agree sale request fields for customer, payment type and medicine quantities with Members 1, 2 and 5.
9. Agree sale response fields for bill identity, totals, batch allocations and customer balance.
10. Document errors for missing medicines, invalid quantities, insufficient stock and unavailable batches.
11. Agree credit-customer requirements and repayment interaction with Member 5.
12. Agree expired and inactive medicine eligibility and same-day expiry treatment with Member 3.
13. Document the earliest-expiry batch allocation rule with a worked example using two batches.
14. Agree deterministic batch order and lock order so concurrent checkouts behave consistently.
15. Agree how checkout avoids duplicate sales after repeated taps, timeouts or uncertain responses.
16. Share API examples and acceptance cases with Member 6 before implementing sales.
17. Review the desktop and mobile POS designs with Member 2.
18. Build the POS shell with medicine search, search results, cart, totals and payment controls.
19. Create the cart-item model and CartProvider using agreed stock units and monetary handling.
20. Implement adding a medicine and combining repeated additions appropriately.
21. Implement quantity increase, decrease, direct editing and item removal.
22. Reject zero, negative or invalid cart quantities and provide clear feedback.
23. Calculate preview subtotals and totals without presenting them as a confirmed sale.
24. Handle an empty cart and prevent payment submission until required data is valid.

Member 4

25. Use clearly identified synthetic data until real API connections are available.
26. Build the Receive Stock form under the stock screens.
27. Add searchable medicine and supplier selectors using shared widgets from Member 2.
28. Add batch number, quantity, cost price, expiry date and received-date fields.
29. Validate required fields, positive quantity, sensible monetary values and future expiry.
30. Display field-specific feedback and keep user input when validation fails.
31. Create the batch service and connect POST /api/batches after Member 3 supplies the endpoint.
32. Add Bearer authorization through the shared API service.
33. Show a loading state and prevent duplicate stock-receipt submission.
34. Display server validation errors without falsely reporting a successful stock receipt.
35. Reset the form only after a confirmed successful receipt.
36. Refresh medicine and batch state after receipt and verify the newly recorded quantity.
37. Test the full receipt form against the stored batch row with Member 6.
38. Create the sales controller and service using the agreed standard response conventions.
39. Validate payment type, cart shape, item IDs and positive quantities before modifying the database.
40. Handle duplicate medicine lines consistently so combined demand cannot bypass stock checks.
41. Require an existing eligible customer when payment type is credit.
42. Fetch authoritative active medicine records and prices from the database.
43. Reject unknown or inactive medicines with a clear error before confirming a sale.
44. Begin one database transaction using one checked-out database connection.
45. Lock eligible stocked batches in the agreed deterministic order.
46. Exclude expired, returned, unavailable and zero-stock batches from allocation.
47. Allocate required quantities from the soonest-expiring eligible batches first.
48. Split a medicine across multiple batches when the first eligible batch is insufficient.
49. Rollback the whole transaction if any medicine has insufficient eligible stock.
50. Create the sales header with the agreed user, customer, payment and monetary details.
51. Insert a sale-item row for each exact batch allocation and preserve the sold unit price.

Member 4

52. Deduct batch quantities and verify they cannot become negative.
53. Apply any agreed discounts on the server using the approved rules.
54. For credit sales, update the customer's balance in the same transaction with appropriate locking.
55. Leave customer credit unchanged for cash sales and failed transactions.
56. Coordinate the post-sale low-stock check with Member 1 so alert changes match committed stock.
57. Commit only when sale, items, stock and credit updates all succeed.
58. On failure, rollback and release the connection before returning a safe error.
59. Return the committed bill, item totals, allocation details and updated credit information as agreed.
60. Implement the agreed duplicate-submission protection for a checkout retry or repeated request.
61. Provide protected sale-detail/history access needed to reopen receipts without making a new sale.
62. Write meaningful tests for single-batch, split-batch and multiple-medicine sales.
63. Test insufficient stock, expired stock, invalid cart, inactive medicine and missing credit customer.
64. Test failure after partial processing and verify all stock, sale and balance changes rollback.
65. Test simultaneous checkouts and confirm no overselling or inconsistent credit balances.
66. Review the transaction implementation with Member 1 and fix issues before merging.
67. Connect live medicine search to the POS using the shared medicine API.
68. Debounce search input and ignore outdated responses so results do not jump incorrectly.
69. Display available-stock information and warn about cart quantities beyond current availability.
70. Connect the cart payment action to the sales service.
71. Add cash and credit payment choices with Member 5's reusable customer picker.
72. Require customer selection for credit and show the selected customer's current balance.
73. Prevent duplicate payment submission while a request is pending.
74. Keep the cart on validation or network failure and explain whether the sale outcome is known.
75. Use the agreed receipt lookup/retry behavior when a timeout leaves the outcome uncertain.
76. Clear the cart only after confirming the sale succeeded.
77. Refresh medicine stock, dashboard and customer credit state after a successful sale.

Member 4

78. Build the receipt screen from the saved sale response rather than recalculating current prices.
79. Show pharmacy identity, bill number, date, line quantities, sold prices and totals.
80. Show payment type and agreed customer/credit details on credit receipts.
81. Implement reopening a previous sale receipt through the sale-detail service.
82. Add Flutter PDF and printing support compatible with Windows and Android.
83. Create a reusable receipt layout for A4 and the pharmacy's actual thermal-paper size if needed.
84. Handle long medicine names and multi-page receipts without clipping or losing totals.
85. Add print, save-PDF and supported share actions with clear cancellation/error behavior.
86. Test receipt totals against sales records for cash, credit, discounts and split-batch sales.
87. Test actual pharmacy printer behavior with authorized staff when a printer is available.
88. Fix checkout and receipt issues found during Member 6's integration and acceptance tests.
89. Document the sale algorithm, API examples and receipt-generation setup for Member 6.
90. Work with Member 3 to define the legacy CSV import template and field mapping.
91. Build the import tool with required-field, quantity, price and expiry validation.
92. Implement dry-run behavior that reports errors without changing production records.
93. Implement agreed duplicate detection and safe re-run behavior for imported rows.
94. Produce accepted/rejected-row summaries and preserve original input privately.
95. Test import failures and ensure incomplete data does not silently corrupt stock relationships.
96. Run the import on a test database and reconcile counts and sample stock with Member 3.
97. Back up the target database before the approved production import.
98. Run the production import with Member 3 after Member 1 completes server setup.
99. Verify imported stock can be searched, received, sold and traced to the correct batches.
100. Perform authorized checkout checks on the installed Windows and Android releases.
101. Provide receipt examples and explain cash/credit checkout during Member 5's staff training.
102. Hand over import instructions, reconciliation results, receipt settings and checkout troubleshooting notes.
103. Confirm sales and migration changes are tested, reviewed, merged and included in the final build.

Member 5

1. Read the proposal and blueprint and identify all customer, credit, birthday, messaging and training responsibilities.
2. Inspect the customer, notification, provider, service and screen locations in the repository.
3. Install the backend, Flutter and API tools needed for the assigned work.
4. Clone the repository and use feature branches and reviewed pull requests.
5. Interview pharmacy staff about customer registration, credit sales, repayment and birthday greetings.
6. Confirm the minimum customer information staff actually need and who may access it.
7. Agree required name/phone validation, optional birthday handling and duplicate-customer rules.
8. Agree phone normalization and search behavior suitable for the pharmacy's customer records.
9. Agree credit-balance meaning, currency precision and the treatment of existing unpaid balances.
10. Agree repayment validation, overpayment policy and any permitted correction process with Members 1 and 6.
11. Confirm whether customers have an app with registered device tokens or require an SMS channel.
12. Confirm the chosen messaging provider, recipient eligibility, cost and approved message content with the owner.
13. Record the notification decision with Member 6 instead of assuming staff-device FCM reaches customers.
14. Agree birthday timezone, leap-day behavior and how to handle failed or missed sends.
15. Coordinate customer, repayment and delivery-log schema with Member 3.
16. Create the chosen messaging project/provider setup and private test credentials.
17. Keep provider keys outside Git and coordinate secure environment configuration with Member 1.
18. Send a test message to an authorized test recipient and record the setup instructions.
19. Agree customer and repayment API fields and response examples with Members 2 and 4.
20. Implement protected customer-create and customer-list endpoints.
21. Validate customer input and normalize phone values before storing or searching.
22. Implement the agreed duplicate-customer handling with clear validation messages.
23. Implement customer search and the agreed pagination behavior for the directory and POS picker.
24. Implement customer-detail and customer-update endpoints.
25. Return clear errors for missing customers and invalid profile changes.

Member 5

26. Apply authentication and role checks so customer information is restricted to authorized staff.
27. Avoid exposing internal messaging credentials or unnecessary private data in customer responses.
28. Create synthetic customers with birthdays, no birthday, zero balance and outstanding credit.
29. Coordinate linked opening-credit or historical data so seeded balances reconcile with the chosen ledger design.
30. Test customer create, search, update, duplicates, missing fields and unauthorized access.
31. Create typed customer and credit/payment models for the Flutter client.
32. Create the customer service using the shared authenticated API service.
33. Create CustomerProvider with load, search, refresh and error state.
34. Build the searchable customer directory with name, phone and outstanding balance.
35. Show unpaid balances clearly without relying only on color.
36. Add loading, no-customers, no-results, retry and server-error states.
37. Build the add-customer form with validated required fields and optional birthday input.
38. Build the edit-customer form and preserve existing values when requests fail.
39. Connect create/update actions and refresh the directory after confirmed success.
40. Create a reusable customer search/picker for Member 4's POS.
41. Provide the selected customer ID and current balance without creating duplicate customer state.
42. Add the has_outstanding_credit flag to customer responses using the agreed balance rules.
43. Implement the outstanding-credit endpoint listing only customers who owe money.
44. Sort debtors by highest balance and support the agreed search/filter behavior.
45. Implement the protected customer-repayment endpoint.
46. Validate positive amount, customer existence and overpayment policy before posting a repayment.
47. Begin one transaction and lock the customer balance or relevant ledger records before changing credit.
48. Insert the repayment record with amount, actor, payment reference and the agreed recorded date.
49. Update the balance atomically and return the confirmed new balance and payment details.
50. Rollback and release the database connection on any repayment failure.
51. Apply the agreed duplicate-submission protection for repeated repayment requests.
52. Expose protected payment history and the last-payment information needed by the credit report.

Member 5

53. Coordinate credit-sale balance updates with Member 4 so sale and repayment locking are compatible.
54. Test partial repayment, exact repayment, invalid amount, unknown customer and unauthorized posting.
55. Test overlapping credit sale and repayment requests and reconcile the resulting balance.
56. Test repeated submission and verify one repayment cannot accidentally reduce the balance twice.
57. Confirm failed credit sales and repayments leave balances and history unchanged.
58. Implement birthday matching by month/day using the agreed pharmacy timezone.
59. Handle missing birthdays and agreed leap-day cases explicitly.
60. Create the configurable scheduled birthday job and register it with Member 1's job setup.
61. Isolate provider-specific sending logic behind a replaceable messaging service.
62. Fetch only eligible recipients and validate token/phone availability before sending.
63. Create the approved birthday message template without including sensitive purchase or credit details.
64. Record delivery attempts/results with customer, occasion and provider reference as agreed.
65. Use durable uniqueness/locking rules so concurrent or repeated jobs do not duplicate the greeting.
66. Record provider failures and define safe retry behavior that does not blindly resend confirmed messages.
67. Handle invalid or expired device tokens according to the selected provider's response.
68. Add a development-only test action restricted to approved test recipients.
69. Ensure the test action and test credentials are not enabled in production.
70. Test successful delivery, missing recipient, provider error, repeated runs and duplicate prevention.
71. Document provider setup, message eligibility, operational limits and troubleshooting steps.
72. Work with Member 4 to add cash/credit payment choices to the POS.
73. Require a selected existing customer before allowing credit checkout.
74. Show agreed outstanding-credit warnings without inventing an unapproved credit limit.
75. Refresh customer balance only after the server confirms a successful sale.
76. Provide updated customer/credit details to the receipt flow and directory.
77. Build the outstanding-credit report screen using desktop and mobile layouts.
78. Show customer name, phone, amount owed, last payment and the combined outstanding total.
79. Add customer search and highest-debt sorting.

Member 5

80. Reconcile report totals with the customer API and database using known fixtures.
81. Add a Record Payment action to each eligible debtor entry.
82. Build the repayment dialog with amount, confirmation and current-balance context.
83. Validate the amount before submitting and show server-side policy errors clearly.
84. Prevent duplicate repayment taps while a request is pending.
85. Refresh customer, payment history and outstanding totals only after confirmed success.
86. Keep the entered payment available for correction when the server rejects it.
87. Test the full sequence of credit sale, partial repayment, final repayment and zero-balance report removal.
88. Review customer and credit changes with Member 1 and client changes with Member 2.
89. Update customer/credit API documentation, messaging setup notes and test evidence for Member 6.
90. Prepare a short user guide covering login, customer registration, cash sale, credit sale, payment and receipt.
91. Capture readable screenshots from the actual final interface with synthetic or authorized example data.
92. Create a one-page staff cheat sheet using pharmacy terminology.
93. Prepare a hands-on training script covering common errors as well as successful operations.
94. Review the guide and message wording with the pharmacy owner and correct confusing language.
95. Help Members 3 and 4 map authorized legacy customer and opening-credit records if they exist.
96. Reconcile imported customer identities and outstanding balances with authorized pharmacy staff.
97. Verify production messaging recipients/settings with the owner before enabling real delivery.
98. Run hands-on training on the installed Windows and Android applications.
99. Ask staff to perform the workflows themselves and record questions and difficulties.
100. Assign feedback to the responsible members and verify corrected customer/credit issues.
101. Help Member 6 execute pharmacy acceptance tests for credit, repayments, customer search and messaging.
102. Hand over the user guide, cheat sheet, provider-operation notes and support contact process.
103. Confirm the owner understands recipient requirements, channel costs and any accepted messaging limitations.
104. Confirm customer, credit and messaging work is tested, reviewed, merged and included in the final release.

Member 6

1. Read both source documents and compile every functional, nonfunctional and security requirement.
2. Inspect the repository structure and identify where requirements, diagrams, API contracts, test evidence and guides belong.
3. Install Git, Postman, database tools and the tools needed for reporting and document preparation.
4. Clone the repository and follow feature-branch and pull-request workflows.
5. Coordinate repository collaborator access and the shared main/develop branch workflow.
6. Configure the agreed pull-request review requirements with the repository administrator.
7. Create a team task board with clear ownership and To Do, Doing, In Review and Done states.
8. Record incomplete tasks, dependencies, defects and decisions without assuming a planned task is complete.
9. Create a requirement-to-owner-to-test traceability register covering FR01-FR20 and all proposal modules.
10. Include credit, password changes, roles, administration, backups and notification requirements in the register.
11. Review feature ownership and identify missing endpoints, screens or integration responsibilities.
12. Document agreed scope and record conditional items such as actual supplier-return mutation or discounts.
13. Capture acceptance criteria for authentication, inventory, sales, alerts, credit, reporting and handover.
14. Create the software requirements specification with purpose, problem, users, objectives, scope and constraints.
15. Describe the pharmacy's existing manual process and the intended digital workflows.
16. Document nonfunctional expectations for usability, reliability, compatibility, response time and data protection.
17. Coordinate unresolved architecture, timezone, stock, money and notification decisions with the assigned owners.
18. Create the shared Postman workspace and Medixmart API collection.
19. Create environment placeholders for base URL and token without committing real credentials.
20. Organize API requests by feature and add reusable authorization handling.
21. Add a login test script that stores a test token in the local Postman environment.
22. Create a test-case template with ID, requirement, preconditions, steps, expected result, actual result and status.
23. Create a defect template with reproduction steps, evidence, severity, owner and retest result.
24. Agree controlled synthetic test data and database-reset instructions with Member 3.
25. Test health behavior and database connectivity errors with Member 1.
26. Test valid login, wrong password, missing credentials and malformed request bodies.

Member 6

27. Test missing, malformed, invalid and expired tokens on protected endpoints.
28. Test Admin/Pharmacist permissions and disabled-account behavior.
29. Test password changes, session behavior and protection against disabling the final usable administrator.
30. Test user/settings administration and access to agreed audit information.
31. Test customer create, list, detail, search, edit, duplicate handling and invalid input.
32. Confirm customer information is unavailable to unauthorized requests.
33. Record authentication/customer results and assign failed cases to their owners.
34. Create the architecture diagram showing Windows/Android clients, Express, PostgreSQL, jobs and messaging.
35. Create the use-case diagram with Admin and Pharmacist actors and their agreed permissions.
36. Create the ER diagram matching actual tables, keys and relationships.
37. Create the class/domain diagram required by the proposal and align it with implemented models.
38. Keep editable diagram sources and export readable versions for the final documents.
39. Write the system-design explanation, including client/server responsibility and deployment topology.
40. Collect agreed endpoint contracts, payload examples, errors and access rules into API documentation.
41. Document transactional sale allocation and why expiry-first behavior differs from arrival-order FIFO.
42. Test category creation, listing, validation and duplicate rules.
43. Test medicine create, edit, search, price/reorder validation and calculated available-stock totals.
44. Test safe medicine deactivation and preservation of historical sale references.
45. Test supplier create, edit, detail, search and invalid references.
46. Test batch creation with valid links, received date, cost and future expiry.
47. Test missing fields, negative/zero quantity, invalid medicine/supplier and nonfuture expiry.
48. Receive stock through the app and verify the exact batch row and stock increase in SQL.
49. Test separate batches of one medicine with different expiry dates and equal-expiry tie cases.
50. Test any agreed audited stock-correction flow and verify invalid changes rollback.
51. Agree sales/stock report and dashboard response fields with Members 2, 3 and 4.
52. Implement protected today-sales reporting with the agreed reporting timezone and date boundary.
53. Implement the seven-day sales series, including days with no sales.
54. Reconcile report calculations against known seeded sale headers/items without double-counting split batches.
55. Share working dashboard report responses with Member 2 for client integration.
56. Test single-batch sales and compare stock, sale header, sale items and receipt totals.

Member 6

57. Test allocation across two or more batches with the earliest-expiry batch exhausted first.
58. Test multiple medicines, duplicate cart lines and insufficient stock in a later cart item.
59. Test expired/inactive medicines, empty cart, invalid quantity and missing credit customer.
60. Force an intermediate sale failure and confirm all stock, sale and credit changes rollback.
61. Submit simultaneous sales and confirm total allocations cannot exceed eligible stock.
62. Test repeated checkout submissions and uncertain-response recovery without creating duplicate bills.
63. Test cash sale behavior and credit sale balance changes against database results.
64. Test partial/full repayment, invalid repayment, overpayment policy and duplicate repayment submission.
65. Test concurrent credit-sale and repayment updates and reconcile the customer balance.
66. Test expiry warnings at the seven-month boundary, already expired stock and zero-stock batches.
67. Test scheduled-job startup catch-up, repeated execution and prevention of duplicate alerts.
68. Test low-stock conditions above, equal to and below the reorder level.
69. Test low-stock warning resolution after replenishment and recurrence after a later shortage.
70. Test alert filters, summary counts, read state and dashboard refresh/drill-down.
71. Test birthday matching, recipient eligibility, leap-day policy, failures and duplicate-send prevention.
72. Confirm the birthday test endpoint cannot be used in production.
73. Test supplier-return filtering and reconcile candidates by supplier, expiry and quantity.
74. If actual returns are enabled, test return history, stock deduction, duplicate submission and rollback.
75. Run an end-to-end flow covering medicine setup, receipt, cash sale, credit sale, repayment, alerts and reports.
76. Record screenshots, request/response evidence and SQL reconciliation for important acceptance cases.
77. Write the development chapter explaining tools, code organization, transactions, expiry-first allocation and jobs.
78. Write the testing chapter with the test plan, real results, defect fixes and unresolved limitations.
79. Implement the date-filtered sales-report endpoint with validated date ranges and agreed totals.
80. Implement the stock-report endpoint using the same stock and expiry definitions as the inventory API.
81. Include the required medicine, category, batch/supplier or status details in the agreed report outputs.
82. Implement CSV export with correct escaping, headings and spreadsheet-safe cell handling.
83. Implement Excel export with readable columns, monetary/date formats and totals.
84. Enforce authentication/authorization on report and export endpoints.
85. Build sales and stock report screens using Member 2's shared responsive widgets.

Member 6

86. Add report filters, date selection, loading, empty and error states.
87. Add Export actions and verify save/open behavior on Windows and Android.
88. Compare every export total and filtered row set with its screen/API and SQL results.
89. Test invalid ranges, no-data periods, long names and larger report results.
90. Verify receipt PDFs, print/save/share actions and reopened receipt accuracy with Member 4.
91. Coordinate accessibility, keyboard, real-device and actual-resolution checks with Member 2.
92. Measure agreed common-operation response times and report slow paths to their owners.
93. Review logs, exports, fixtures and documentation for accidental secrets or private customer data.
94. Run an internal acceptance script using the application as pharmacy staff would.
95. Prioritize defects, assign owners, retest fixes and keep the traceability register current.
96. Verify database backup/restore evidence with Member 1 and reconcile restored counts and balances.
97. Review migration dry-run results, rejected rows and stock/customer-credit reconciliation with Members 3, 4 and 5.
98. Prepare the pharmacy acceptance script and confirm expected outcomes with the owner.
99. Verify release installation, clean-device builds, restart behavior and Android-to-server connectivity.
100. Run acceptance tests on the actual installed system with authorized pharmacy staff.
101. Record the owner's feedback, accepted limitations and remaining blocking defects.
102. Retest corrected defects on the final release and obtain written UAT sign-off when acceptance criteria pass.
103. Finalize the SRS, design, development, testing, deployment and recovery documentation.
104. Integrate Member 5's user guide and training evidence into the handover pack.
105. Check final screenshots, diagrams, API examples and instructions match the delivered application.
106. Prepare the defense presentation covering problem, solution, architecture, demonstration, tests, challenges and future work.
107. Prepare a reliable demonstration or backup demo recording using nonprivate data.
108. Run a practice presentation with all members and prepare answers about stack choice, stock allocation and security.
109. Collect versioned build references, editable documents, schema/setup instructions and final test evidence.
110. Confirm the pharmacy received the installer/APK, guides, training, backup instructions and support details.
111. Check all requirement coverage and recorded limitations before final submission.
112. Confirm all final work is reviewed, merged and included in the project handover package.